

10 장 학습 과제

과제 1. 함수 템플릿과 타입 요구 조건

`GetMin()`, `GetMax()`와 `Clamp()` 함수 템플릿을 작성한다.

요구 사항:

- `int`, `double`과 사용자 정의 `Level` 타입에서 동작
- `Level`은 비교 연산을 제공
- 혼합 타입 호출이 실패하는 이유 설명
- 명시적 템플릿 인수로 혼합 타입을 호출했을 때 발생하는 변환 확인
- `const T&` 반환형에서 임시 객체를 전달할 때의 수명 문제 분석

함수가 실제로 요구하는 연산과 의미 조건을 표로 정리한다.

과제 2. 배열 크기 추론

`C` 배열을 참조로 받아 원소 수를 반환하는 `ArraySize()`를 작성한다. 원소 타입과 크기를 각각 `T`, `Size`로 추론해야 한다.

추가 기능:

- 배열의 모든 원소를 출력하는 `PrintArray()`
- 배열의 합계를 반환하는 `SumArray()`
- 빈 배열을 만들 수 없는 `C++` 배열의 특성 설명
- 배열을 포인터 매개변수로 받았을 때 크기 정보가 사라지는 이유 설명

`static_assert`로 여러 크기의 결과를 검증한다.

과제 3. `FixedArray` 클래스 템플릿

원소 타입과 크기를 템플릿 인수로 받는 `FixedArray<T, Size>`를 구현한다.

필수 기능:

- 값 초기화된 원소 저장
- `At()`의 상수·비상수 오버로드
- `operator[]`의 상수·비상수 오버로드
- `Fill()`
- `Size()`
- 범위 오류 시 `std::out_of_range`
- `Size > 0` 컴파일 시점 검사

`FixedArray<int, 4>`와 `FixedArray<int, 8>`이 서로 다른 타입임을 `std::is_same_v`로 확인한다.

`Mermaid` 클래스 다이어그램에 타입 매개변수와 비타입 매개변수를 표시한다.

과제 4. 클래스 템플릿 인수 추론

값 하나를 저장하는 `ValueBox<T>`와 두 값을 저장하는 `PairBox<First, Second>`를 구현한다.

요구 사항:

- 생성자 인수로 CTAD 적용
- 문자열 리터럴을 `std::string`으로 저장하는 추론 가이드 작성
- 명시적 템플릿 인수와 추론 결과 비교
- 추론 가이드가 일반 함수가 아닌 이유 설명
- 복사 생성에서 어떤 추론 후보가 사용되는지 확인

컴파일러가 추론한 타입을 `static_assert`로 검증한다.

과제 5. 전체 특수화와 부분 특수화

`TypeDescription<T>` 클래스 템플릿을 작성한다.

분류할 타입:

- 일반 값 타입
- 포인터 타입
- 고정 크기 배열 타입
- `bool` 전체 특수화
- `const T` 형태

각 특수화는 타입 범주 이름을 반환해야 한다. 여러 부분 특수화가 동시에 일치할 수 있는 사례를 만들고 모호성이 발생하는 이유를 설명한다. 같은 기능을 타입 특성과 `if constexpr`로 작성한 방식과 비교한다.

과제 6. 가변 인수 로그 함수

가변 인수 템플릿과 폴드 표현식으로 로그 함수를 작성한다.

필수 기능:

- 로그 수준 출력
- 인수 개수 출력
- 여러 타입의 값을 순서대로 출력
- 각 값 사이에 구분자 삽입
- 인수가 없는 호출 처리
- 출력할 수 없는 타입에서 이해하기 쉬운 컴파일 오류 제공

출력 가능성을 검사하는 `StreamWritable Concept`를 정의한다. 재귀적 전개 버전과 폴드 표현식 버전의 코드 길이와 인스턴스화 구조를 비교한다.

과제 7. 게임 객체 **Concept**

상속 관계가 없는 `Player`, `Monster`, `BreakableBox`를 작성한다. 세 타입은 체력 또는 내구도 상태를 자체적으로 관리한다.

`Damageable Concept` 요구 조건:

- `TakeDamage(int)` 호출 가능
- 결과를 `int`로 변환 가능
- `GetHp()` 또는 동일한 의미의 상태 조회 함수 제공
- `IsDead()`가 `bool` 결과 제공

`ApplyDamage()`와 `PrintHealth()`를 `Concept`로 제약한다. 요구 조건을 만족하지 않는

`Decoration` 타입을 전달했을 때 컴파일 오류를 확인한다. 동적 인터페이스 `IDamageable` 방식과 `Concept` 방식의 UML을 각각 작성하고 사용 시점을 비교한다.

과제 8. 타입 특성과 `if constexpr`

여러 타입의 값을 문자열로 설명하는 `DescribeValue()`를 작성한다.

구분할 범주:

- 정수형
- 부동소수점형
- 열거형
- 포인터
- 문자열
- 그 밖의 출력 가능한 객체

`std::is_integral_v`, `std::is_floating_point_v`, `std::is_enum_v`, `std::is_pointer_v`, `std::is_same_v`와 `std::remove_cvref_t`를 사용한다. 널 포인터를 안전하게 처리하고 선택되지 않은 분기가 인스턴스화되지 않는 사례를 포함한다.

과제 9. SFINAE 코드를 Concepts로 변경

`std::enable_if_t`를 사용하여 정수형과 부동소수점형의 `Normalize()` 오버로드를 작성한다. 같은 인터페이스를 C++20 Concepts로 다시 작성한다.

비교 항목:

- 함수 선언의 가독성
- 오류 메시지 위치
- 오버로드 후보 선택
- 조건 재사용 방식
- 구현 내부에서 타입 특성이 여전히 필요한 부분

기존 SFINAE 코드를 유지해야 하는 상황과 Concepts로 전환할 수 있는 상황을 구분한다.

과제 10. 완벽 전달 기반 게임 객체 팩토리

`GameObject` 추상 클래스와 `Player`, `Monster`, `Projectile` 구체 클래스를 작성한다. 생성자 매개변수의 타입과 개수는 서로 달라야 한다.

`CreateGameObject<T>(Args&&...)` 요구 사항:

- T가 `GameObject`의 공개 파생 타입인지 검사
- 지정한 인수로 T를 생성할 수 있는지 검사
- 생성자 인수를 완벽 전달
- `std::unique_ptr<GameObject>` 반환
- 전역 상태 사용 금지
- 잘못된 생성자 인수에서 컴파일 시점 오류 발생

복사 횟수와 이동 횟수를 출력하는 추적 타입을 만들어 원값과 오른값 전달 결과를 비교한다. `std::move`를 잘못 사용한 팩토리과 `std::forward`를 사용한 팩토리의 차이를 확인한다.

종합 UML에서 관계 표현확인:

- `GameObject`와 구체 클래스의 상속
- 각 객체가 소유하는 컴포넌트
- 팩토리 함수 템플릿의 생성 의존
- `Concept`가 요구하는 정적 인터페이스

실행 시간 다형성과 정적 다형성을 어느 위치에 사용했는지 문장으로 설명한다.